

Pauta de Evaluación — Proyecto Final: Diseño de Compilador

Autómatas y Compiladores · Escuela de Ingeniería Informática · PUCV · Semestre 1 – 2026

Cada ítem se evalúa por niveles de logro (no 0/1). La última columna indica qué debes producir y cómo se comprobará. Total: 100 puntos, convertidos a nota 1.0–7.0 con 60% de exigencia (4.0 al 60%, 7.0 al 100%).

Ítem	Criterio	Niveles de logro y puntaje	Cómo se comprueba (evidencia)	Pts
A. Diseño y originalidad del lenguaje				10
A1	Originalidad del lenguaje	0: copia evidente del lenguaje visto en clases · 2: solo variaciones cosméticas · 4: lenguaje propio pero genérico · 6: lenguaje original, con propósito y sintaxis creativos y justificados.	Tu lenguaje debe verse claramente distinto al de clases: palabras clave, símbolos y propósito propios. Incluye al inicio del ejemplo (o del notebook) 2–3 líneas que expliquen la temática o el propósito. Se comprueba comparando tus palabras clave y estructura con las del lenguaje visto en clases.	6
A2	Coherencia del diseño	0: sintaxis ambigua o inconsistente · 2: parcialmente coherente · 4: sintaxis consistente, palabras clave y reglas uniformes en toda la gramática.	Usa siempre la misma forma para la misma estructura (si un bloque va entre llaves, que sea así en todos; si las sentencias terminan en ';', que terminen todas). Se comprueba leyendo el EBNF y el ejemplo y buscando que no haya formas contradictorias para una misma construcción.	4
B. Léxico y sintaxis — características del lenguaje				32
B1	Tipos de datos (≥3)	0: menos de 2 tipos · 3: 2 tipos funcionando · 5: 3 tipos declarados pero uso limitado · 7: ≥3 tipos declarados y usados correctamente en el ejemplo.	En tu ejemplo declara y usa al menos 3 tipos distintos (p. ej. entero, flotante, texto o booleano), cada uno con una operación real (asignar, imprimir u operar). Se comprueba ubicando las reglas de tipo en el .g4 y ejecutando el ejemplo para ver que cada tipo se declara y se usa sin error.	7
B2	Estructuras de control (1 condicional + ≥2 repetitivas)	0: ninguna · 3: solo condicional o solo 1 repetitiva · 5: condicional + 1 repetitiva · 6: condicional + 2 repetitivas que parsean · 8: condicional + ≥2 repetitivas que ejecutan correctamente.	Tu ejemplo debe incluir 1 condicional y al menos 2 estructuras repetitivas distintas (p. ej. mientras y para), cada una haciendo algo observable (imprimir o cambiar un valor). Se comprueba ejecutando el ejemplo y viendo que el condicional toma la rama correcta y que cada ciclo repite las veces esperadas.	8
B3	Operadores matemáticos (+, −, *, /)	0: ninguno · 2: 2–3 operadores · 3: los 4 parsean · 4: los 4 con precedencia y resultado correctos.	Incluye en el ejemplo una expresión que combine +, −, * y /, e imprime su resultado. Se comprueba ejecutando y comparando con el valor correcto, atendiendo a la precedencia (p. ej. 2 + 3 * 4 debe dar 14, no 20).	4
B4	Operadores lógicos (≥2)	0: ninguno · 2: 1 operador · 3: 2 operadores que parsean · 4: ≥2 operadores con evaluación correcta (V/F).	Usa al menos 2 operadores lógicos (p. ej. AND, OR, NOT) dentro de la condición de un 'si' o un ciclo, e imprime algo que dependa del resultado. Se comprueba ejecutando con valores que hagan la condición verdadera y falsa, y viendo que se toma la rama correcta en cada caso.	4
B5	Impresión de datos	0: no existe · 2: imprime pero limitado a un tipo · 3: imprime distintos tipos correctamente.	Tu ejemplo debe imprimir valores de distinto tipo (p. ej. un número y un texto) usando la instrucción de impresión de tu lenguaje. Se comprueba ejecutando el ejemplo y viendo que la salida en pantalla es la esperada para cada tipo.	3
B6	Funciones matemáticas (≥4)	0: ninguna · 2: 1–2 funciones · 4: 3 funciones · 6: ≥4 funciones (sqrt, sin, cos, ...) con resultado correcto.	Invoca en el ejemplo al menos 4 funciones matemáticas (p. ej. raíz, seno, coseno, potencia) e imprime cada resultado. Se comprueba ejecutando y comparando cada salida con el valor correcto (p. ej. raíz(9) = 3).	6
C. Análisis semántico (3 reglas a elección)				18

Ítem	Criterio	Niveles de logro y puntaje	Cómo se comprueba (evidencia)	Pts
C1	Regla semántica 1	0: ausente · 2: declarada pero no detecta el caso · 4: detecta el caso · 6: detecta + mensaje de error claro y ubicado (línea/identificador).	Junto al ejemplo válido, entrega un caso pequeño que VIOLE esta regla (puede ir comentado o en un archivo aparte). Se comprueba corriendo ese caso y verificando que el compilador lo detecta y muestra un mensaje claro que diga qué falló y dónde (línea o identificador).	6
C2	Regla semántica 2	0: ausente · 2: declarada pero no detecta el caso · 4: detecta el caso · 6: detecta + mensaje de error claro y ubicado.	Junto al ejemplo válido, entrega un caso pequeño que VIOLE esta segunda regla. Se comprueba corriendo ese caso y verificando que el compilador lo detecta y muestra un mensaje claro indicando qué falló y dónde.	6
C3	Regla semántica 3	0: ausente · 2: declarada pero no detecta el caso · 4: detecta el caso · 6: detecta + mensaje de error claro y ubicado.	Junto al ejemplo válido, entrega un caso pequeño que VIOLE esta tercera regla. Se comprueba corriendo ese caso y verificando que el compilador lo detecta y muestra un mensaje claro indicando qué falló y dónde.	6
D. Transpilación a Python y ejecución (Fase 4)				12
D1	Transpila y ejecuta	0: no transpila · 3: genera Python pero no ejecuta (errores) · 5: transpila y ejecuta con salida parcialmente correcta · 7: transpila y ejecuta el ejemplo con la salida esperada completa.	El notebook debe, para tu ejemplo: (1) imprimir el código Python generado y (2) ejecutarlo con <code>exec()</code> mostrando la salida. Se comprueba corriendo todas las celdas en orden y comparando esa salida con la que esperarías de tu programa fuente.	7
D2	Correctitud de la traducción (puntos críticos)	Por aspectos correctos en el Python generado — 0: la mayoría falla · 2: 2 aspectos · 3: 3 aspectos · 5: los 5: indentación de bloques anidados; lógicos traducidos ($y \rightarrow \text{and}$, $o \rightarrow \text{or}$); cabeceras terminan en <code>':'</code> ; funciones a <code>math.*</code> ; precedencia con paréntesis.	Incluye un ejemplo con un bloque anidado (un <code>'si'</code> dentro de un ciclo), un operador lógico y una función matemática. Se comprueba mirando el Python generado: indentación de los anidados correcta, lógicos como <code>and/or</code> , cabeceras terminadas en <code>':'</code> , funciones como <code>math.*</code> y precedencia conservada con paréntesis.	5
E. Orden y calidad del código				12
E1	Estructura y modularidad	0: todo en un bloque, código espagueti · 2: poca separación · 4: etapas separadas (lexer / parser / semántico / main) con nombres significativos · 6: además sin duplicación ni código muerto, responsabilidades claras.	Organiza el notebook en secciones o funciones separadas por etapa (lexer, parser, semántico, generador, main), con nombres que digan qué hacen. Se comprueba abriendo el notebook y verificando que se distingue cada etapa y que no hay código repetido ni celdas sin uso.	6
E2	Legibilidad	0: indentación caótica, nombres crípticos · 3: legible pero con inconsistencias · 6: indentación consistente, nombres descriptivos y funciones de tamaño razonable.	Mantén indentación consistente, nombres de variables y funciones descriptivos, y funciones que quepan en pantalla (evita bloques enormes). Se comprueba leyendo el notebook: debe poder seguirse sin esfuerzo y sin nombres tipo <code>x1</code> , <code>aux</code> o <code>temp</code> sin sentido.	6
F. Documentación				10
F1	Instrucciones de ejecución	0: no se explica cómo correrlo · 2: instrucciones incompletas · 3: explica cómo ejecutar · 4: README/celda inicial explica ejecución y describe el lenguaje.	Agrega al inicio del notebook (o un README) una celda que explique en qué consiste el lenguaje y los pasos exactos para ejecutarlo: qué celdas correr, en qué orden y dónde poner el código de ejemplo. Se comprueba siguiendo esas instrucciones desde cero y logrando ejecutar sin adivinar.	4
F2	Comentarios pertinentes	0: sin comentarios o solo ruido · 1: comentarios mínimos · 2: comentarios en código o gramática · 3: comentarios que explican decisiones clave en .g4 y notebook.	Comenta las partes no obvias del .g4 y del notebook explicando el porqué (no el qué): decisiones de diseño, reglas tramposas, mapeos a Python. Se comprueba revisando que los comentarios ayuden a entender y no sean ruido (evita <code>'# suma dos números'</code> sobre algo evidente).	3
F3	Ejemplo documentado	0: ejemplo sin explicación · 1: explica parcialmente · 2: comentado · 3: comentado y ejerce las características del lenguaje.	El ejemplo .txt debe llevar comentarios que expliquen qué demuestra cada parte y ejercitar las características del lenguaje (tipos, control, operadores, funciones). Se comprueba leyéndolo: debe entenderse qué hace y qué característica prueba cada bloque.	3

Ítem	Criterio	Niveles de logro y puntaje	Cómo se comprueba (evidencia)	Pts
G. Entregables y formato				6
G1	Gramática EBNF (.txt)	0: ausente o incorrecta · 1: presente con errores de notación · 2: presente, bien formada en E-BNF y consistente con la .g4.	Entrega la gramática en notación E-BNF en un .txt, y que coincida con lo que realmente implementa tu .g4. Se comprueba abriendo el .txt, verificando que la notación E-BNF está bien formada y que sus reglas corresponden a las del .g4.	2
G2	Código de ejemplo (.txt)	0: ausente · 1: presente pero pobre · 2: presente e ilustrativo del lenguaje.	Entrega un .txt con código fuente en tu lenguaje que sea representativo (que use varias características, no una línea trivial). Se comprueba abriéndolo y verificando que es un programa válido y no trivial en tu lenguaje.	2
G3	.zip con notebook + .g4	0: falta algún archivo · 1: completo pero con problemas de carga · 2: completo y carga/ejecuta sin errores de estructura.	Entrega un .zip que contenga el notebook de Colab y los archivos .g4, sin archivos de más. Se comprueba descomprimiendo y abriendo el notebook en Colab: debe cargar y poder ejecutarse sin que falten archivos.	2
BONIFICACIÓN — más allá de los mínimos (no sube el máximo de 100)				6
Z1	Características extra que funcionan	0: nada extra · 1: una característica adicional al mínimo que ejecuta (tipo, estructura de control u operador) · 2: dos o más, integradas y ejecutándose.	Tu ejemplo usa y ejecuta correctamente característica(s) por sobre el mínimo exigido. Se comprueba ejecutando el ejemplo y viendo que esa(s) característica(s) adicional(es) funciona(n).	2
Z2	Funciones o reglas semánticas extra	0: nada extra · 1: una función matemática (>4) o regla semántica (>3) adicional que funciona · 2: dos o más adicionales funcionando.	Invoca/activa la función o regla adicional. Se comprueba ejecutándola: la función extra entrega el valor correcto, o la regla semántica extra detecta su caso con mensaje claro.	2
Z3	Ambición técnica o creatividad	0: nada · 1: un extra logrado (mejores mensajes de error en todo el pipeline, o salida/experiencia mejorada) · 2: algo ambicioso y pulido (p. ej. modo interactivo/REPL, o una característica de lenguaje propia no pedida, bien lograda).	Muestra el extra en el notebook o en una celda de demostración. Se comprueba ejecutándolo y verificando que aporta valor real (no es decorativo) y que funciona.	2

Política de evaluación

- Bonificación: premia ir más allá de los mínimos; suma puntos pero el total nunca pasa de 100 (tope nota 7.0). Sirve para asegurar el 7.0 o recuperar puntos perdidos en otro ítem.
- Atraso: se descuenta por ventanas de 30 minutos. Apenas se pasa la hora de corte ya corre la 1ª ventana (entregar a las 00:00 ya penaliza) y cada 30 minutos adicionales suman otra (se redondea hacia arriba). El descuento se acumula.
- Corte: pasadas 24 horas de la hora de corte, la entrega no se recibe (nota 1.0).
- El atraso descuenta del puntaje total, no de un criterio: mide cumplimiento, no la calidad del trabajo.